



API Versioning



Pedro Constantino



Repost

Swip





Configure Version

```
builder.Services.AddApiVersioning(options =>
{
    options.DefaultApiVersion = new ApiVersion(1);
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.ReportApiVersions = true;
    options.ApiVersionReader = ApiVersionReader.Combine(
        new QueryStringApiVersionReader("v"),
        new HeaderApiVersionReader("X-API-Version")
    );
});
```

You can adjust the settings inside the delegate according to your needs

```
builder.Services.AddVersionedApiExplorer(options =>
{
    options.GroupNameFormat = "'v'V";
    options.SubstituteApiVersionInUrl = true;
});
```

By adding it, it automatically fixes the endpoint routes and substitutes the API version route parameter.



Pedro Constantino



Repost

Swip





API Versioning in Controller APIs

```
[ApiController]
[ApiVersion(1)]
[ApiVersion(2)]
[Route("api/v{version:apiVersion}/users")]
public class UsersController : ControllerBase
{
    ...
}
```

API versioning is essential for keeping your system adaptable and ensuring smooth upgrades

```
[HttpGet("{id}")]
[MapToApiVersion(1)]
public IActionResult GetUserByIdV1(int id)
```

```
[HttpGet("{id}")]
[MapToApiVersion(2)]
public IActionResult GetUserByIdV2(int id)
```

Maps each method to its respective version.



Pedro Constantino



Repost

Swip





API Versioning in Minimal APIs

`WithApiVersion` is used to explicitly map an API version to a specific endpoint.



```
app.MapGet("/api/v{version:apiVersion}/users", (ApiVersion version) =>
{
    ...
}).WithApiVersion(1);

// or

}).WithApiVersion(2);
```

We can use `ApiVersionSet` to group API versions and apply versioning centrally.



```
}).WithApiVersionSet(apiVersionSet).WithApiVersion(2);
```



Pedro Constantino



Repost

Swip



Thanks for your attention.

**Let's connect and spread
knowledge together.**



Pedro Constantino



Repost